Trees
树

Chapter Summary

- Introduction to Trees
- Applications of Trees (*not currently included in overheads*)
- Tree Traversal
- Spanning Trees
- Minimum Spanning Trees (*not currently included in overheads*)

Introduction to Trees

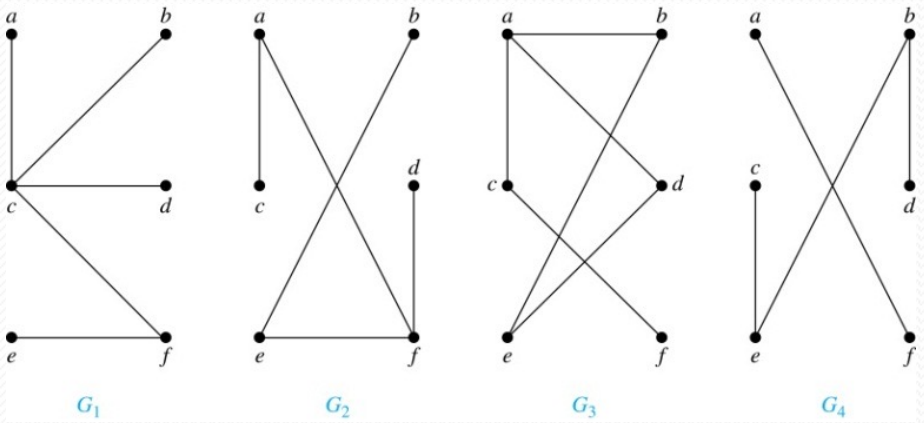
Section Summary

- Introduction to Trees
- Rooted Trees
- Trees as Models
- Properties of Trees

Trees

Definition: A *tree* is a connected undirected graph with no simple circuits.
定义：树是没有简单回路的连通无向图。

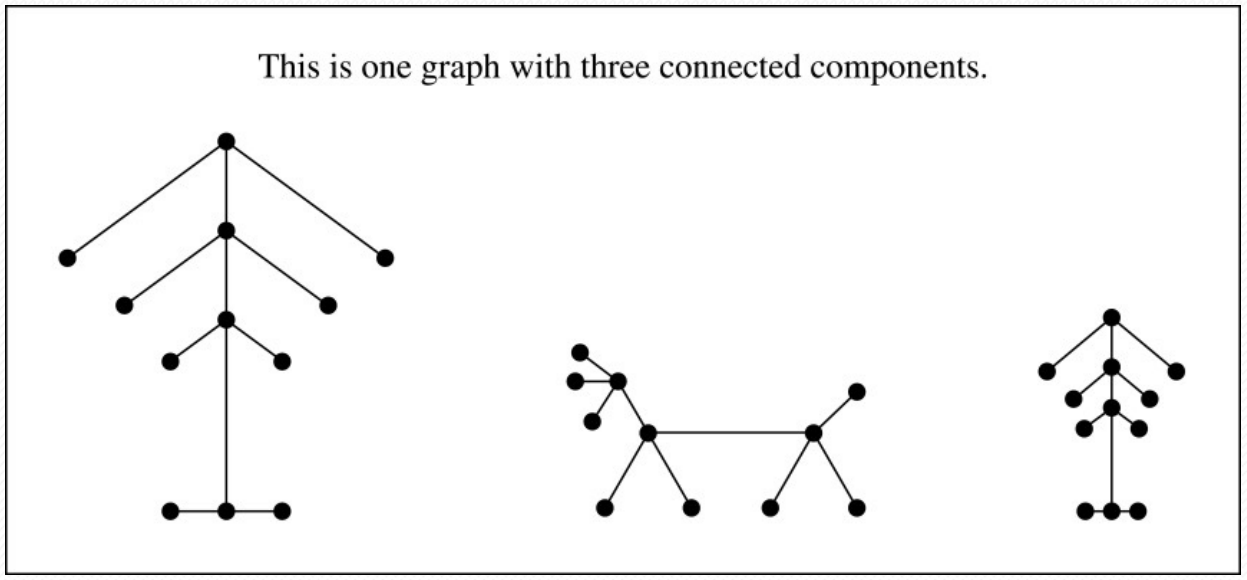
Example: Which of these graphs are trees?



Solution: G_1 and G_2 are trees - both are connected and have no simple circuits. Because *e*, *b*, *a*, *d*, *e* is a simple circuit, G_3 is not a tree. G_4 is not a tree because it is not connected.

Definition: A *forest* is a graph that has no simple circuit, but is not connected. Each of the connected components in a forest is a tree.

定义：森林是不含简单回路但不一定连通的图。每个连通分支都是树。



Trees (*continued*)

Theorem: An undirected graph is a tree if and only if there is a unique simple path between any two of its vertices.

定理：一个无向图是树当且仅当它的每对顶点之间存在唯一简单通路。

必要性：若 G 是树，则任意两点之间存在唯一简单通路

因为 G 是树，所以按照定义，G 是**连通且无回路**的无向图。

任取两个不同顶点 u, v。

由于 G 连通，所以 u 与 v 之间至少存在一条通路。把这条通路中重复出现的顶点删去，就可以得到一条从 u 到 v 的**简单通路**。因此，u, v 之间至少存在一条简单通路。

下面证明这条简单通路唯一。

假设 u 与 v 之间存在两条不同的简单通路：

$$P_1 : u \rightsquigarrow v, \qquad P_2 : u \rightsquigarrow v.$$

因为这两条通路不同，所以它们在某些边或顶点上必有差异。沿着 P_1 和 P_2 比较，从 u 出发，取它们第一次分开的顶点为 x，再取它们之后第一次重新相遇的顶点为 y。

则在 P_1 中，x 到 y 的部分是一条通路；在 P_2 中，x 到 y 的部分也是一条通路。并且这两段通路不同，除端点 x, y 外没有共同顶点。

于是这两段通路合在一起形成一个回路，这与 G 是树、即无回路矛盾。

所以 u 与 v 之间的简单通路唯一。

因此，树中任意两点之间存在唯一简单通路。

Theorem: An undirected graph is a tree if and only if there is a unique simple path between any two of its vertices.

定理：一个无向图是树当且仅当它的每对顶点之间存在唯一简单通路。

充分性：若任意两点之间存在唯一简单通路，则 G 是树

现在假设 G 的任意两个不同顶点之间存在唯一简单通路。

首先，因为任意两点之间都存在简单通路，所以 G 是**连通图**。

下面证明 G 无回路。

反设 G 中存在一个回路：

$$v_0 v_1 v_2 \cdots v_k v_0,$$

其中 $k \geq 2$，且 $v_0, v_1, \ldots, v_k$ 两两不同。

考虑顶点 v_0 和 v_1。

一方面，边 $v_0 v_1$ 本身就是一条从 v_0 到 v_1 的简单通路：

$$v_0 \to v_1.$$

另一方面，沿着回路的另一方向，也有一条从 v_0 到 v_1 的简单通路：

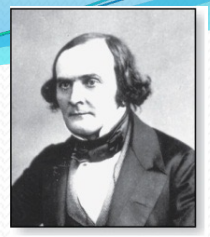
$$v_0 \to v_k \to v_{k-1} \to \cdots \to v_2 \to v_1.$$

这两条简单通路不同，矛盾于“任意两点之间存在唯一简单通路”。

所以 G 中不存在回路。

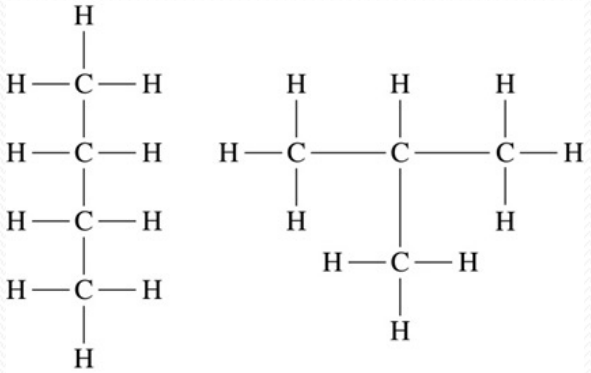
因此，G 既连通又无回路，所以 G 是树。

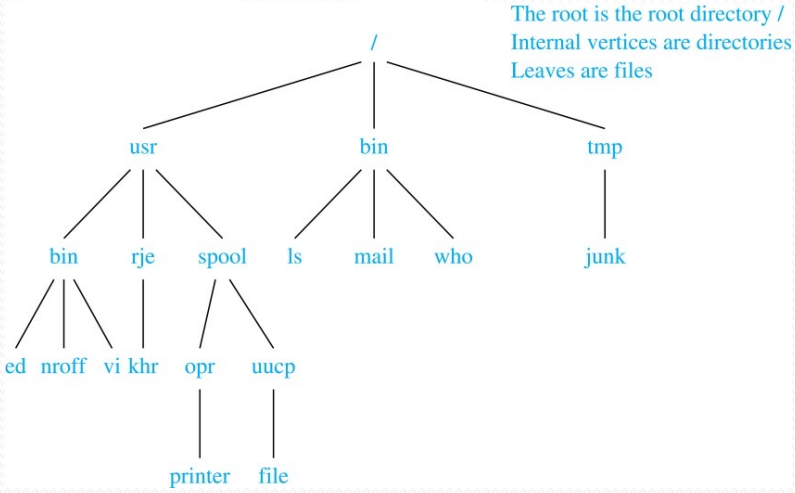
Arthur Cayley
(1821-1895)

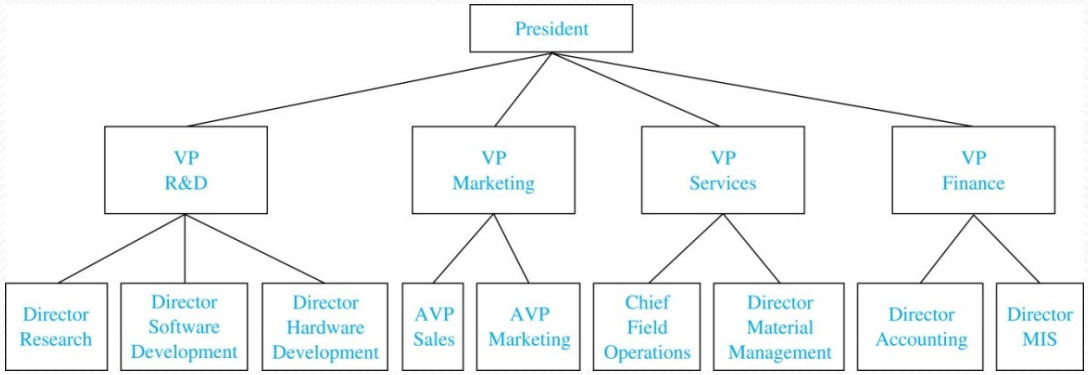


Trees as Models

- Trees are used as models in computer science, chemistry, geology, botany, psychology, and many other areas.
- Trees were introduced by the mathematician Cayley in 1857 in his work counting the number of isomers of saturated hydrocarbons. The two isomers of butane are shown at the right.
- The organization of a computer file system into directories, subdirectories, and files is naturally represented as a tree.
- Trees are used to represent the structure of organizations.





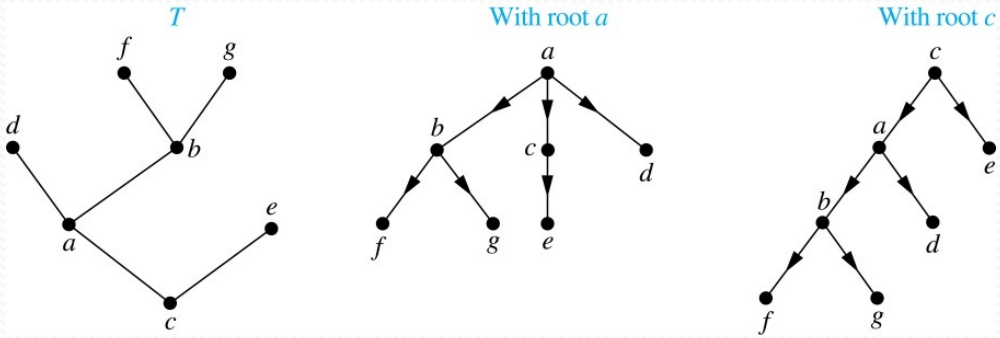


Rooted Trees

Definition: A *rooted tree* is a tree in which one vertex has been designated as the *root* and every edge is directed away from the root.

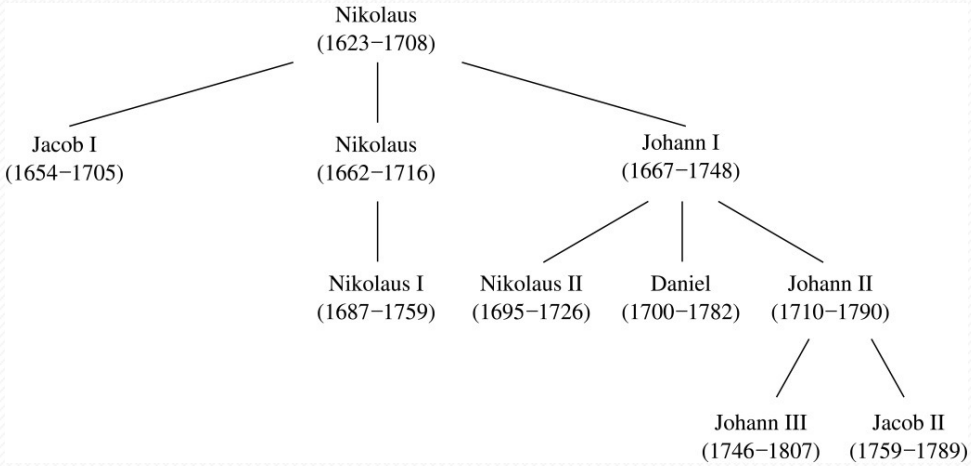
定义：**根树**是指定一个顶点作为根并且每条边的方向都离开根的树。

An unrooted tree is converted into different rooted trees when different vertices are chosen as the root.



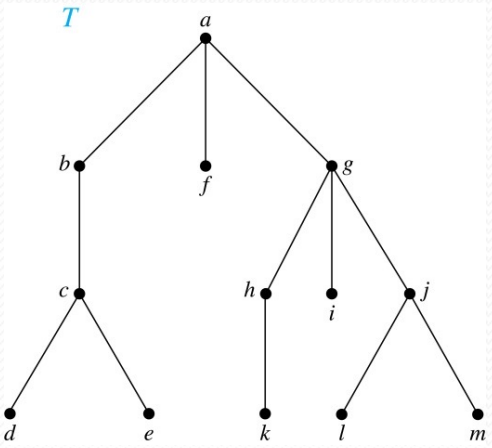
Rooted Tree Terminology

- Terminology for rooted trees is a mix from botany and genealogy (such as this family tree of the Bernoulli family of mathematicians).



- If *v* is a vertex of a rooted tree other than the root, the *parent* of *v* is the unique vertex *u* such that there is a directed edge from *u* to *v*. When *u* is a parent of *v*, *v* is called a *child* of *u*. Vertices with the same parent are called *siblings*.
- The *ancestors* of a vertex are the vertices in the path from the root to this vertex, excluding the vertex itself and including the root. The *descendants* of a vertex *v* are those vertices that have *v* as an ancestor.
- A vertex of a rooted tree with no children is called a *leaf*. Vertices that have children are called *internal vertices*.
- If *a* is a vertex in a tree, the *subtree* with *a* as its root is the subgraph of the tree consisting of *a* and its descendants and all edges incident to these descendants.

Terminology for Rooted Trees



Example: In the rooted tree T (with root a):

(i) Find the parent of c, the children of g, the siblings of h, the ancestors of e, and the descendants of b.

(ii) Find all internal vertices and all leaves.

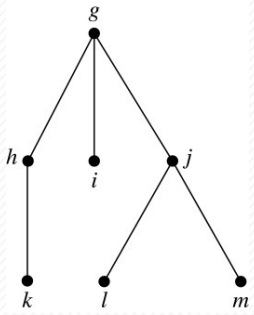
(iii) What is the subtree rooted at G?

Solution:

(i) The parent of c is b. The children of g are h, i, and j. The siblings of h are i and j. The ancestors of e are c, b, and a. The descendants of b are c, d, and e.

(ii) The internal vertices are a, b, c, g, h, and j. The leaves are d, e, f, i, k, l, and m.

(iii) We display the subtree rooted at g.

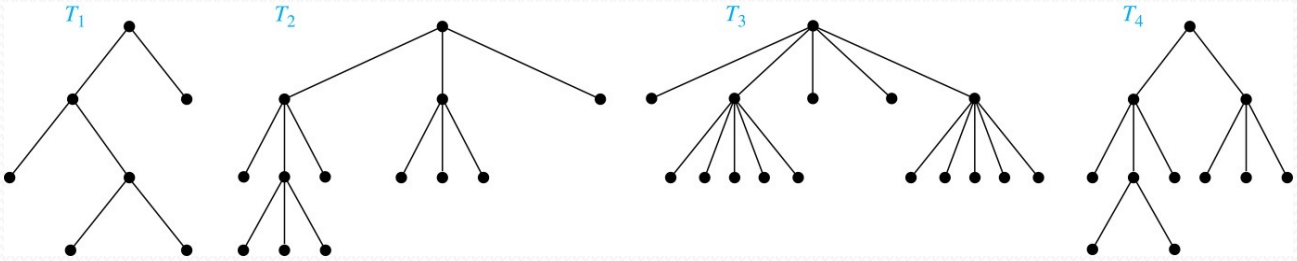


m-ary Rooted Trees (*m*元树)

Definition: A rooted tree is called an *m-ary tree* if every internal vertex has no more than *m* children. The tree is called a *full m-ary tree* if every internal vertex has exactly *m* children. An *m*-ary tree with $m = 2$ is called a *binary* tree.

定义：若根树的每个内点都有不超过*m*个子女，则称它为**m元树**。若该树的每个内点都恰好有*m*个子女，则称为**正则*m*元树**。把$m = 2$ 的正则*m*元树称为**二叉树**。

Example: Are the following rooted trees full *m*-ary trees for some positive integer *m*?



Solution: T_1 is a full binary tree because each of its internal vertices has two children. T_2 is a full 3-ary tree because each of its internal vertices has three children. In T_3 each internal vertex has five children, so T_3 is a full 5-ary tree. T_4 is not a full *m*-ary tree for any m because some of its internal vertices have two children and others have three children.

Ordered Rooted Trees

Definition: An *ordered rooted tree* is a rooted tree where the children of each internal vertex are ordered.

有序根树是把每个内点的子女们都排序的根树。

- We draw ordered rooted trees so that the children of each internal vertex are shown in order from left to right.

画有序根树时，以从左到右的顺序来显示每个内点的子女们。

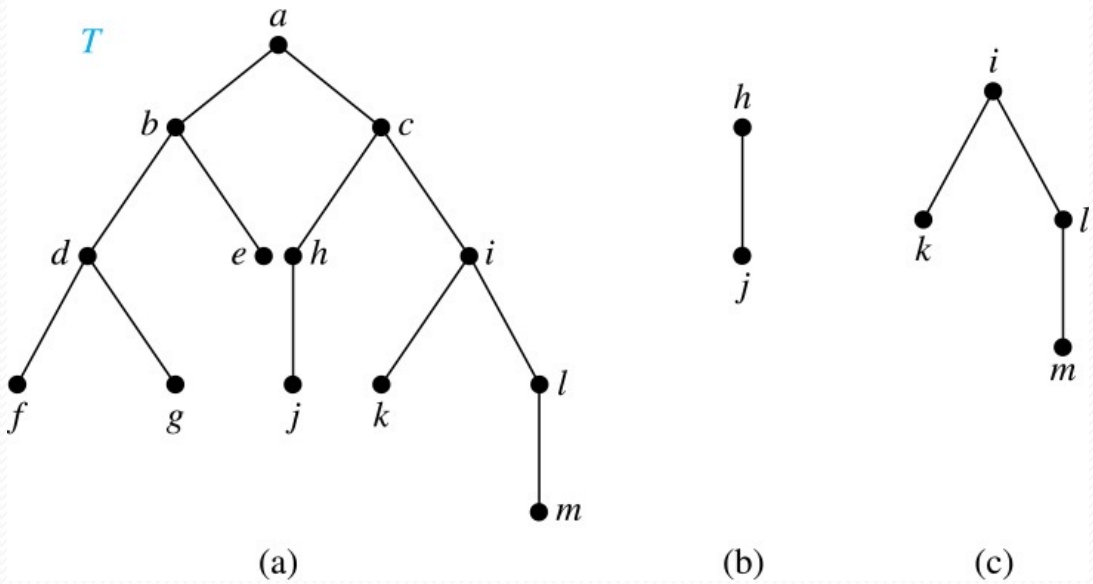
Definition: A *binary tree* is an ordered rooted where where each internal vertex has at most two children. If an internal vertex of a binary tree has two children, the first is called the *left child* and the second the *right child*. The tree rooted at the left child of a vertex is called the *left subtree* of this vertex, and the tree rooted at the right child of a vertex is called the *right subtree* of this vertex.

在有序二叉树中，若一个内点有两个子女，则第一个子女称为左子女而第二个子女称为右子女。根处在一个顶点的左子女处的树称为该顶点的**左子树**，而根处在一个顶点的右子女处的树称为该顶点的**右子树**。

Example: Consider the binary tree T.

(*i*) What are the left and right children of d?

(*ii*) What are the left and right subtrees of c?



Solution:

(*i*) The left child of d is f and the right child is g.

(*ii*) The left and right subtrees of c are displayed in (b) and (c).

Properties of Trees 树的性质

Theorem 2: A tree with n vertices has $n - 1$ edges.

定理2：带有n个顶点的树含有$n-1$条边。

***Proof* (*by mathematical induction*):**

BASIS STEP: When $n = 1$, a tree with one vertex has no edges. Hence, the theorem holds when $n = 1$.

INDUCTIVE STEP: Assume that every tree with k vertices has $k - 1$ edges.

Suppose that a tree T has $k + 1$ vertices and that v is a leaf of T. Let w be the parent of v. Removing the vertex v and the edge connecting w to v produces a tree T' with k vertices. By the inductive hypothesis, T' has $k - 1$ edges. Because T has one more edge than T', we see that T has k edges. This completes the inductive step. ◀

Counting Vertices in Full *m*-Ary Trees

Theorem 3: A full *m*-ary tree with *i* internal vertices has $n = mi + 1$ vertices.

定理 3:带有*i*个内点的正则*m*元树含有$n = mi + 1$顶点。

Proof : Every vertex, except the root, is the child of an internal vertex. Because each of the *i* internal vertices has *m* children, there are *mi* vertices in the tree other than the root. Hence, the tree contains $n = mi + 1$ vertices.

Counting Vertices in Full *m*-Ary Trees (*continued*)

Theorem 4: A full m-ary tree with

(*i*) n vertices has $i = (n-1)/m$ internal vertices and $l = [(m-1)n+1]/m$ leaves,

(*ii*) i internal vertices has $n = mi+1$ vertices and $l = (m-1)i+1$ leaves,

(*iii*) l leaves has $n = (ml-1)/(m-1)$ vertices and $i = (l-1)/(m-1)$ internal vertices.

一个正则**m**元树若带有**n**个顶点，

则带有$i = (n-1)/m$个内点和$l = [(m-1)n+1]/m$个树叶；

i个内点，则带有$n = mi+1$个顶点和$l = (m-1)i+1$个树叶；

l个树叶，则带有$n = (ml-1)/(m-1)$个顶点和$i = (l-1)/(m-1)$个内点。

proofs of parts (ii) and (iii) are left as exercises

- ***Proof (of part i):*** Solving for i in $n = mi + 1$ (from Theorem 3) gives $i = (n - 1)/m$. Since each vertex is either a leaf or an internal vertex, $n = l + i$. By solving for l and using the formula for i, we see that

$$l = n - i = n - (n - 1)/m = [(m - 1)n + 1]/m. \quad \blacktriangleleft$$

Level of vertices and height of trees

- When working with trees, we often want to have rooted trees where the subtrees at each vertex contain paths of approximately the same length.
- To make this idea precise we need some definitions:
  - The *level* of a vertex *v* in a rooted tree is the length of the unique path from the root to this vertex.

在根树里顶点*v*的层是从根到这个顶点的唯一通路的长度。

根的层定义为**0**.

  - The *height* of a rooted tree is the maximum of the levels of the vertices.

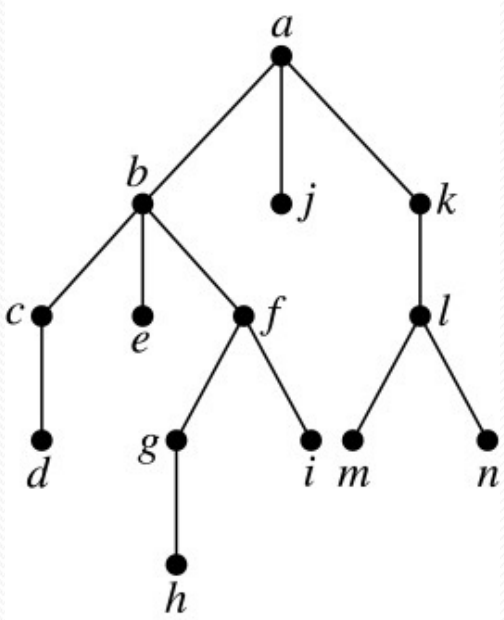
根树的高度就是顶点层数的最大值。

或者说根树的层数是从根到任意顶点的最长通路的长度。

Example:

(*i*) Find the level of each vertex in the tree to the right.

(*ii*) What is the height of the tree?



Solution:

(*i*) The root *a* is at level 0. Vertices *b*, *j*, and *k* are at level 1.

Vertices *c*, *e*, *f*, and *l* are at level 2. Vertices *d*, *g*, *i*, *m*, and *n* are at level 3.

Vertex *h* is at level 4.

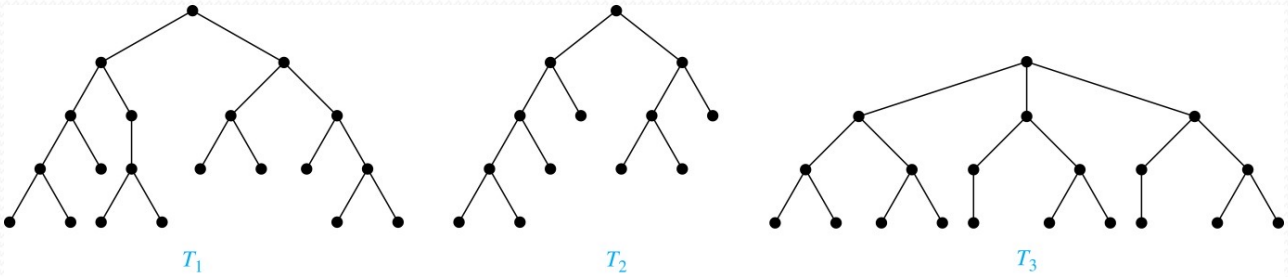
(*ii*) The height is 4, since 4 is the largest level of any vertex.

Balanced *m*-Ary Trees

Definition: A rooted *m*-ary tree of height *h* is *balanced* if all leaves are at levels *h* or $h - 1$.

定义：若一颗高度为*h*的*m*元树的所有顶点都在*h*层或*h-1*层，则这棵树是平衡的。

Example: Which of the rooted trees shown below is balanced?



Solution: T_1 and T_3 are balanced, but T_2 is not because it has leaves at levels 2, 3, and 4.

The Bound for the Number of Leaves in an *m*-Ary Tree

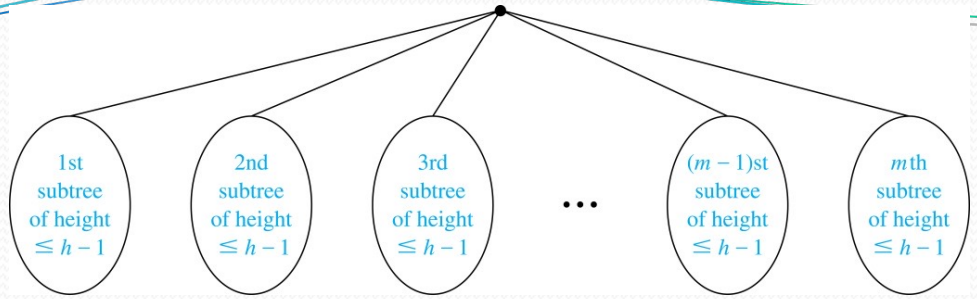
Theorem 5: There are at most m^h leaves in an *m*-ary tree of height *h*.

定理 5：在高度为*h*的*m*元树里至多有 m^h 个树叶。

***Proof* (*by mathematical induction on height*):**

BASIS STEP: Consider an *m*-ary trees of height 1. The tree consists of a root and no more than *m* children, all leaves. Hence, there are no more than $m^1 = m$ leaves in an *m*-ary tree of height 1.

INDUCTIVE STEP: Assume the result is true for all *m*-ary trees of height < *h*. Let *T* be an *m*-ary tree of height *h*. The leaves of *T* are the leaves of the subtrees of *T* we get when we delete the edges from the root to each of the vertices of level 1.



Each of these subtrees has height $\leq h-1$. By the inductive hypothesis, each of these subtrees has at most m^{h-1} leaves. Since there are at most m such subtees, there are at most $m \cdot m^{h-1} = m^h$ leaves in the tree.

Corollary 1: If an m-ary tree of height h has l leaves, then $h \geq \lceil \log_m l \rceil$. If the m-ary tree is full and balanced, then $h = \lceil \log_m l \rceil$. (*see text for the proof*)

推论1: 若一个高度为*h*的*m*元树带有*l*个树叶，则 $h \geq \lceil \log_m l \rceil$。若这个*m*元树是正则的和平衡的，则 $h = \lceil \log_m l \rceil$。

Tree Traversal
树的遍历

Section Summary

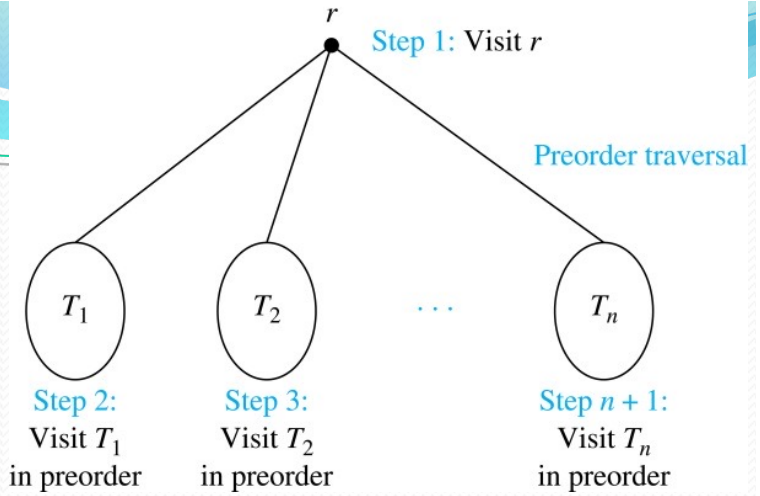
- Universal Address Systems (*not currently included in overheads*)
- Traversal Algorithms
- Infix, Prefix, and Postfix Notation

Tree Traversal

- Procedures for systematically visiting every vertex of an ordered tree are called *traversals*.

  系统地访问有序根树每个顶点的过程都称为遍历算法。
- The three most commonly used *traversals* are *preorder traversal*, *inorder traversal*, and *postorder traversal*.
- 三个最常用的遍历算法：前序遍历、中序遍历和后序遍历。

Preorder Traversal

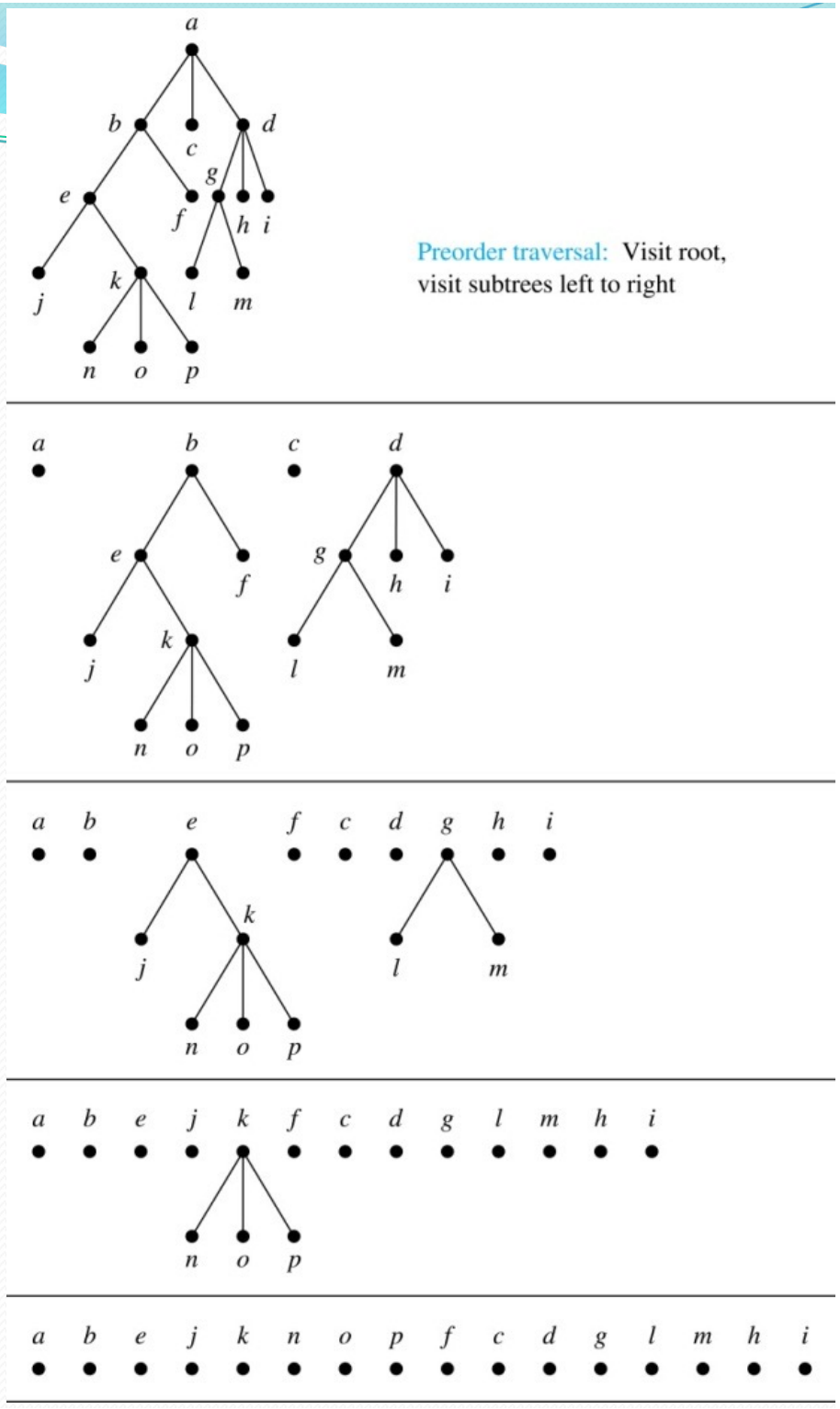


Definition: Let T be an ordered rooted tree with root r. If T consists only of r, then r is the *preorder traversal* of T. Otherwise, suppose that T_1, T_2, ..., T_n are the subtrees of r from left to right in T. The preorder traversal begins by visiting r, and continues by traversing T_1 in preorder, then T_2 in preorder, and so on, until T_n is traversed in preorder.

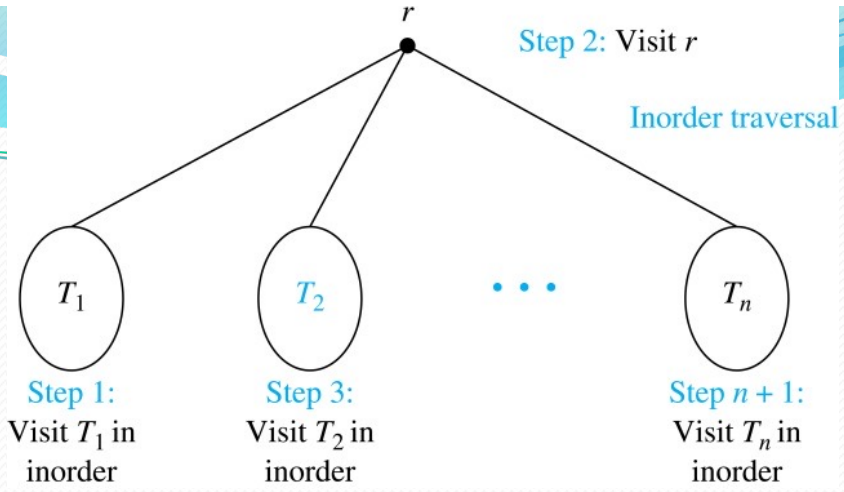
定义1：设T是带根r的有序根树。若T只包含r，则r是T的前序遍历。否则，假定T_1, T_2, ..., T_n 是T里在r处从左向右的子树。**前序遍历**首先访问r，接着以前序来遍历T_1，然后以前序来遍历T_2，依此类推，直到以前序来遍历了T_n 为止。

Preorder Traversal

procedure *preorder* (*T*: ordered rooted tree)
r := root of *T*
list *r*
for each child *c* of *r* from left to right
    T(*c*) := subtree with *c* as root
    preorder(*T*(*c*))



Inorder Traversal

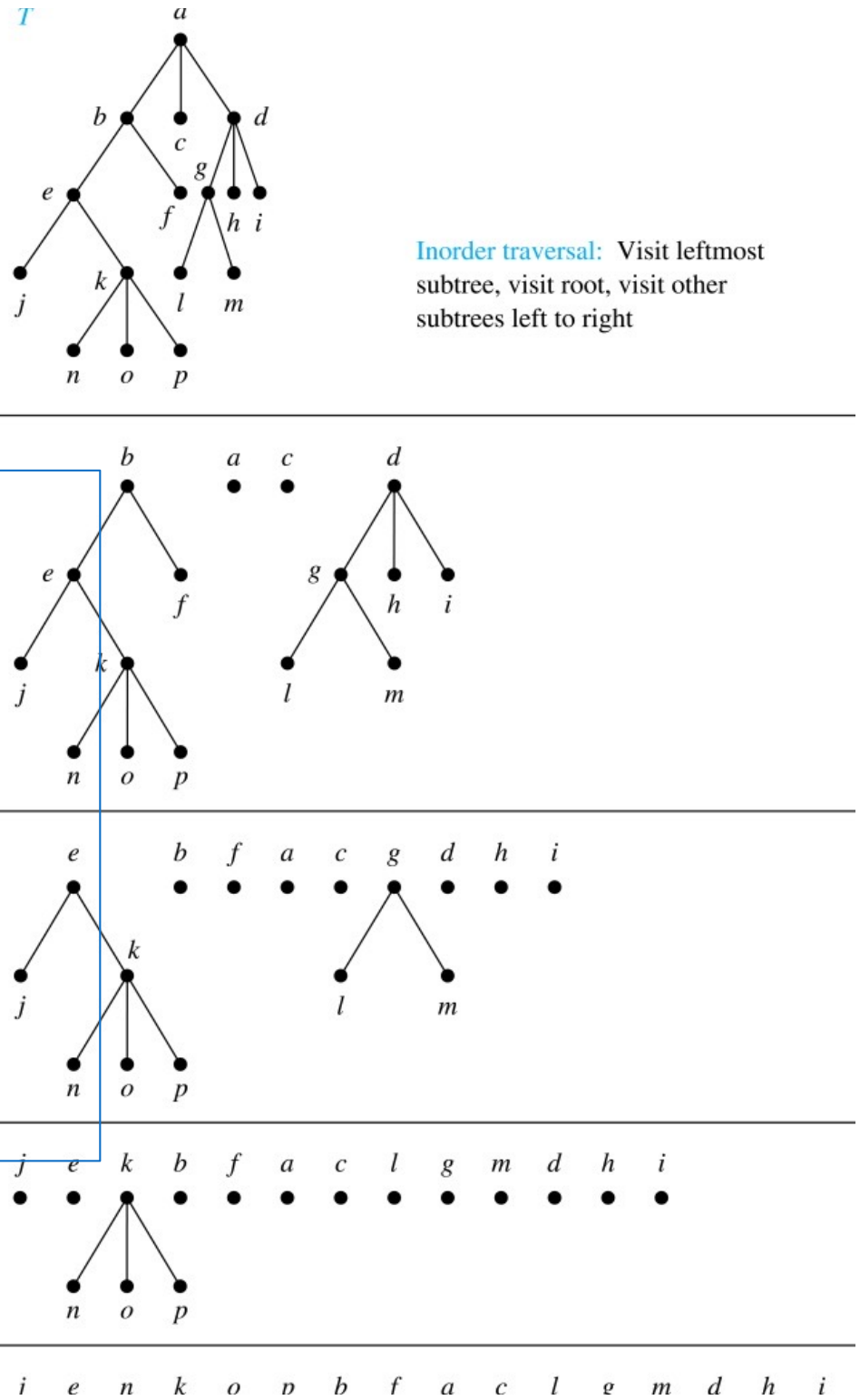


Definition: Let T be an ordered rooted tree with root r. If T consists only of r, then r is the *inorder traversal* of T. Otherwise, suppose that $T_1, T_2, \ldots, T_n$ are the subtrees of r from left to right in T. The inorder traversal begins by traversing T_1 in inorder, then visiting r, and continues by traversing T_2 in inorder, and so on, until T_n is traversed in inorder.

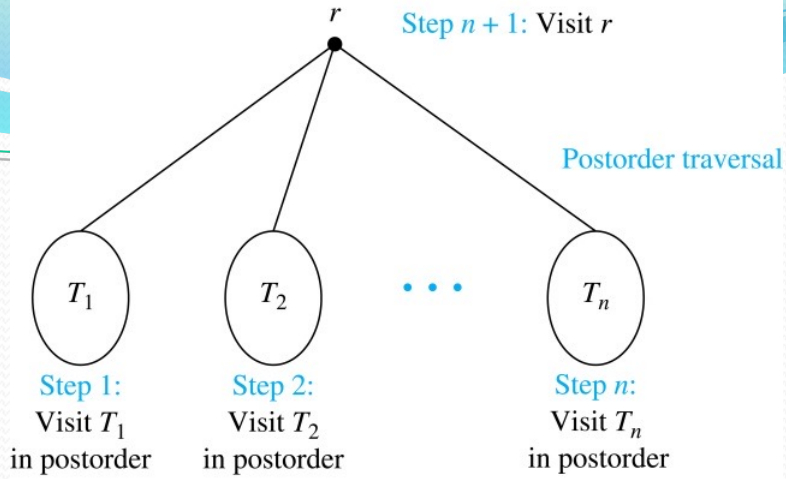
定义2：设 T 是带根r的有序根树。若 T 只包含r，则r是 T 的**中序遍历**。否则，假定$T_1, T_2, \ldots, T_n$ 是 T 里在r处从左向右的子树。**中序遍历**首先以中序来遍历T_1，然后访问r。它接着以中序来遍历T_2，中序遍历T_3，依此类推，直到以中序来遍历了T_n为止。

Inorder Traversal

```
procedure inorder (T: ordered rooted tree)
r := root of T
if r is a leaf then list r
else
    l := first child of r from left to right
    T(l) := subtree with l as its root
    inorder(T(l))
    list(r)
    for each child c of r from left to right
        T(c) := subtree with c as root
        inorder(T(c))
```



Postorder Traversal

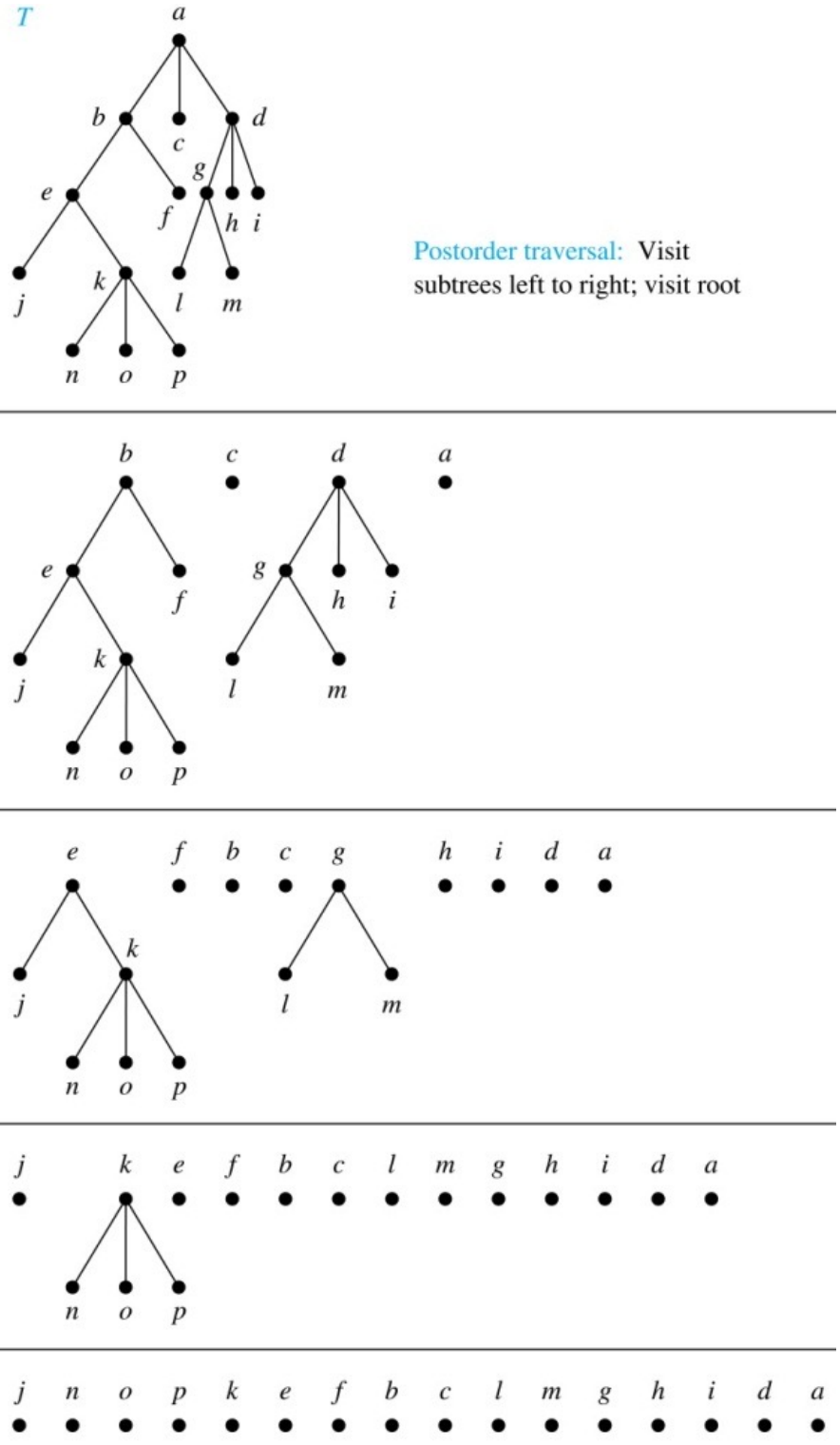


Definition: Let T be an ordered rooted tree with root r. If T consists only of r, then r is the *postorder traversal* of T. Otherwise, suppose that T_1, T_2, ..., T_n are the subtrees of r from left to right in T. The postorder traversal begins by traversing T_1 in postorder, then T_2 in postorder, and so on, after T_n is traversed in postorder, r is visited.

定义3：设 T 是带根r的有序根树。若 T 只包含r，则r是 T 的**后序遍历**。否则，假定$T_1, T_2, ..., T_n$ 是 T 里在r处从左向右的子树。**后序遍历**首先以后序来遍历T_1，然后以后序来遍历T_2，...，然后以后序遍历T_n，最后访问r。

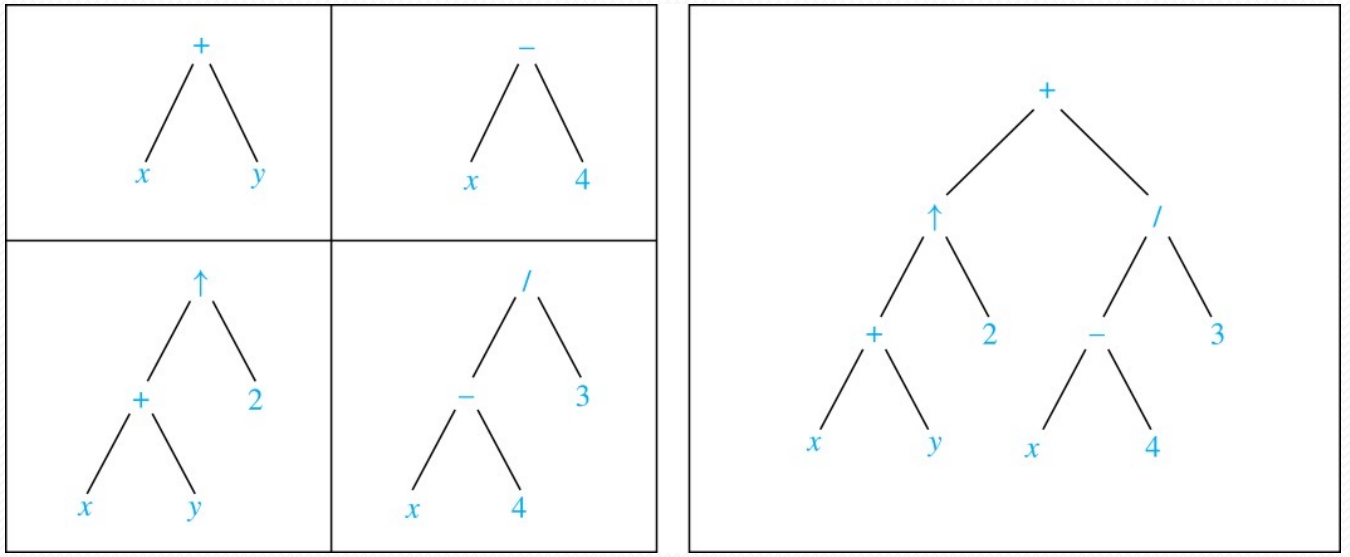
Postorder Traversal

procedure *postordered* (*T*: ordered rooted tree)
r := root of *T*
for each child *c* of *r* from left to right
    T(*c*) := subtree with *c* as root
    postorder(*T*(*c*))
list *r*



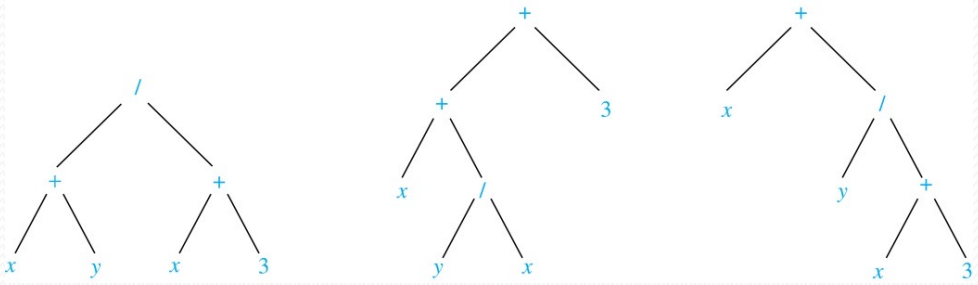
Expression Trees

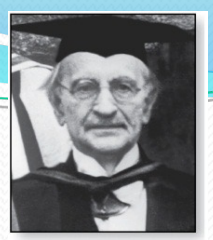
- Complex expressions can be represented using ordered rooted trees.
- 有序树可以用来表示复杂的表达式。
- Consider the expression $((x + y) \uparrow 2) + ((x - 4)/3)$.
- A binary tree for the expression can be built from the bottom up, as is illustrated here.



Infix Notation中缀记法

- An inorder traversal of the tree representing an expression produces the original expression when parentheses are included except for unary operations, which now immediately follow their operands.
- We illustrate why parentheses are needed with an example that displays three trees all yield the same infix representation.





Jan Łukasiewicz
(1878-1956)

Prefix Notation前缀记法

- When we traverse the rooted tree representation of an expression in preorder, we obtain the *prefix* form of the expression. Expressions in prefix form are said to be in *Polish notation*, named after the Polish logician Jan Łukasiewicz.
- Operators precede their operands in the prefix form of an expression. Parentheses are not needed as the representation is unambiguous.
- The prefix form of $((x + y) \uparrow 2) + ((x - 4)/3)$ is $+ \uparrow + x\ y\ 2\ /\ -\ x\ 4\ 3$.
- Prefix expressions are evaluated by working from right to left. When we encounter an operator, we perform the corresponding operation with the two operations to the right.

Example: We show the steps used to evaluate a particular prefix expression:

+ − * 2 3 5 / ↑ 2 3 4

2↑3 = 8

+ − * 2 3 5 / 8 4

8 / 4 = 2

+ − * 2 3 5 2

2 * 3 = 6

+ − 6 5 2

6 − 5 = 1

+ 1 2

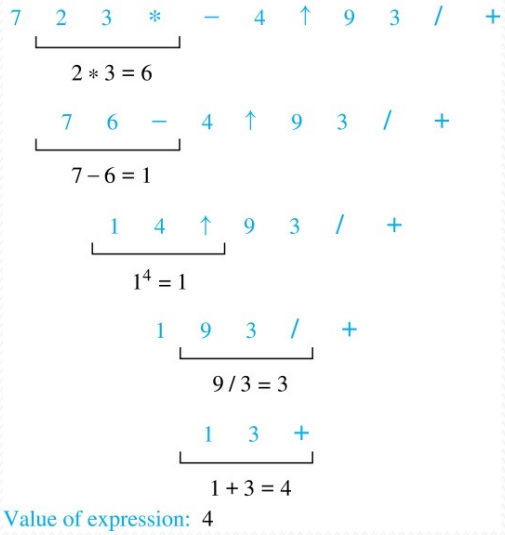
1 + 2 = 3

Value of expression: 3

Postfix Notation后缀记法

- We obtain the *postfix form* of an expression by traversing its binary trees in postorder. Expressions written in postfix form are said to be in *reverse Polish notation*.
- Parentheses are not needed as the postfix form is unambiguous.
- $x\ y + 2 \uparrow x\ 4 - 3 / +$ is the postfix form of $((x + y) \uparrow 2\ ) + ((x - 4)/3)$.
- A binary operator follows its two operands. So, to evaluate an expression one works from left to right, carrying out an operation represented by an operator on its preceding operands.

Example: We show the steps used to evaluate a particular postfix expression.

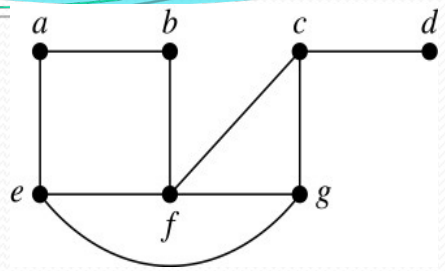


Spanning Trees
生成树

Section Summary

- Spanning Trees
- Depth-First Search
- Breadth-First Search
- Backtracking Applications (*not currently included in overheads*)
- Depth-First Search in Directed Graphs

Spanning Trees

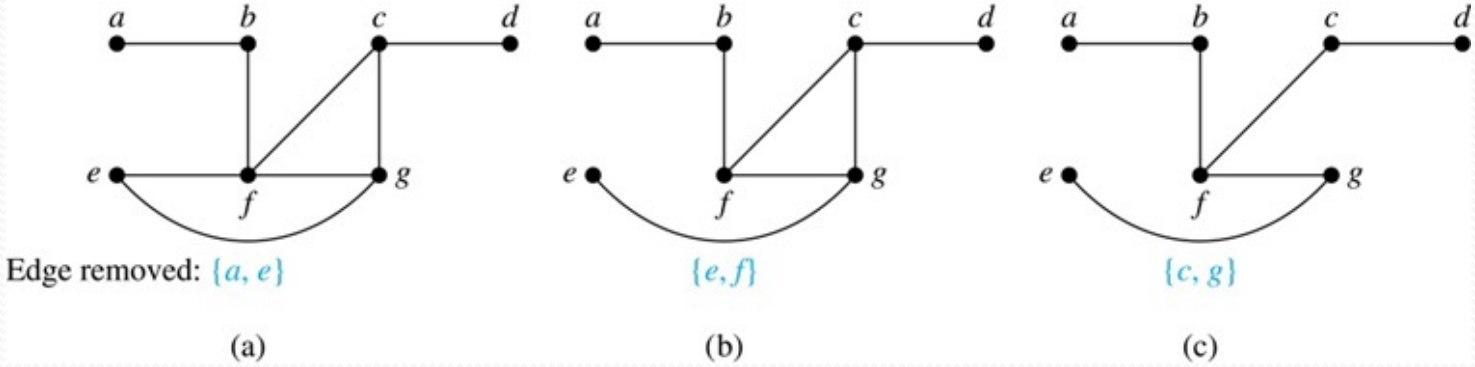


Definition: Let G be a simple graph. A spanning tree of G is a subgraph of G that is a tree containing every vertex of G.

定义：设G 是简单图。G 的生成树是包含G 的每个顶点的G 的子图，且子图必须是树。

Example: Find the spanning tree of this simple graph:

Solution: The graph is connected, but is not a tree because it contains simple circuits. Remove the edge $\{a, e\}$. Now one simple circuit is gone, but the remaining subgraph still has a simple circuit. Remove the edge $\{e, f\}$ and then the edge $\{c, g\}$ to produce a simple graph with no simple circuits. It is a spanning tree, because it contains every vertex of the original graph.



- **定理：简单图是连通的当且仅当它具有生成树。**
- 证明：首先，假定简单图G有生成树T。T包含G的每个顶点。另外，在T的任何两个顶点之间都有T里的通路。因为T是G的子图，所以在G的任何两个顶点之间都有T里的通路。因此，G是连通的。
- 现在假定G是连通的。若G不是树，则它必然包含简单回路。从这些简单回路中的一个里删除一条边。所得出的子图少了一条边，但是仍然包含G的所有顶点并且是连通的。若这个子图不是树，则它有简单回路；所以像前面那样，删除一个简单回路里的一条边。重复这个过程直到没有简单回路为止。这是可能的，因为在图里只有有穷的边数。当没有简单回路剩下时，这个过程终止。产生出一个树，因为在删除边时这个图保持连通。这个树是生成树，因为它包含G的每个顶点。

Spanning Trees (*continued*)

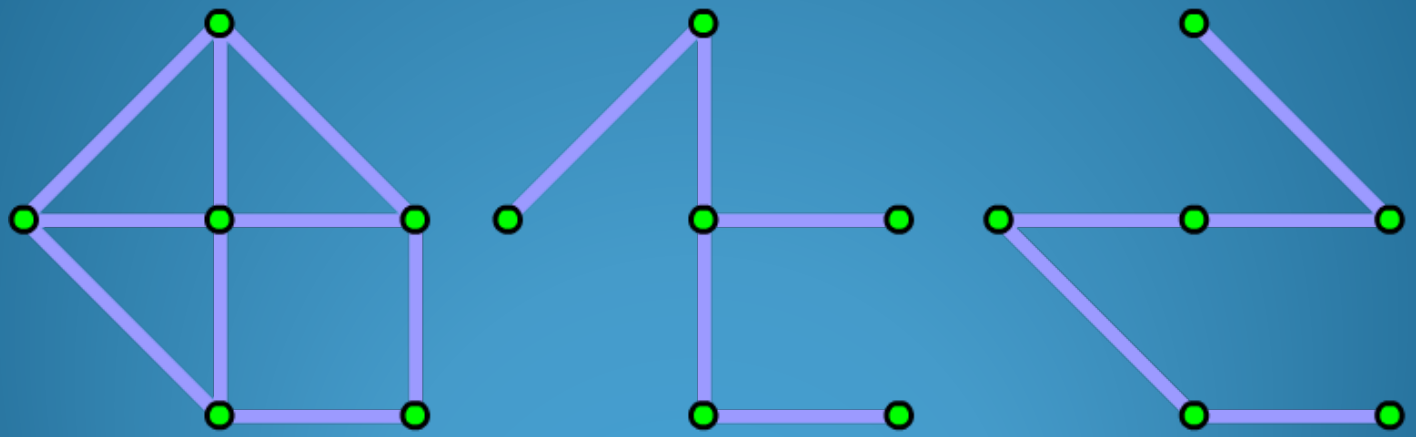
Theorem: A simple graph is connected if and only if it has a spanning tree.

Proof: Suppose that a simple graph G has a spanning tree T. T contains every vertex of G and there is a path in T between any two of its vertices. Because T is a subgraph of G, there is a path in G between any two of its vertices. Hence, G is connected.

Now suppose that G is connected. If G is not a tree, it contains a simple circuit. Remove an edge from one of the simple circuits. The resulting subgraph is still connected because any vertices connected via a path containing the removed edge are still connected via a path with the remaining part of the simple circuit. Continue in this fashion until there are no more simple circuits. A tree is produced because the graph remains connected as edges are removed. The resulting tree is a spanning tree because it contains every vertex of G.

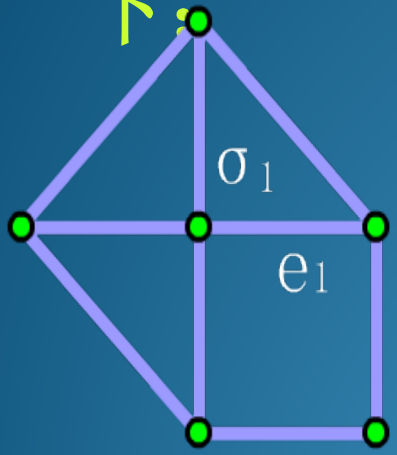
生成树：若连通图G的生成子图T是一棵树，则称T为G的生成树。

下图中（b）和（c）都是（a）的生成树。

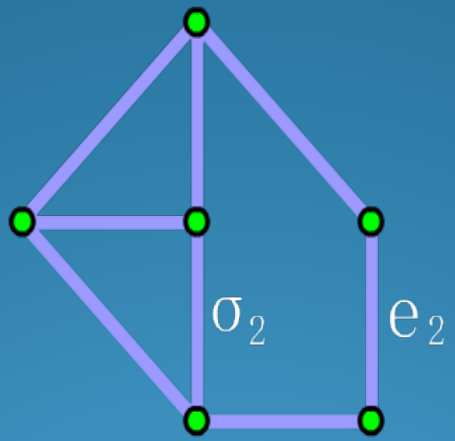


构造连通图G=（V，E）的生成树的方法

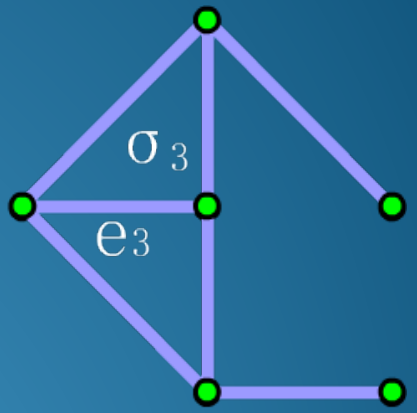
（a）破环法，构造上页图（a）的生成树的过程如下：



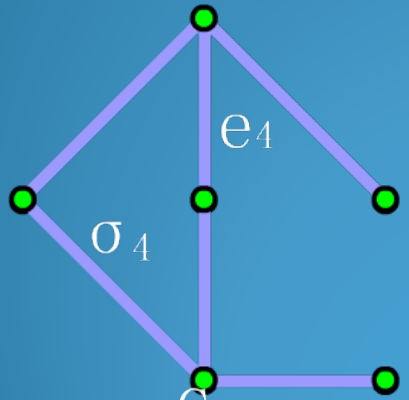
G=G1



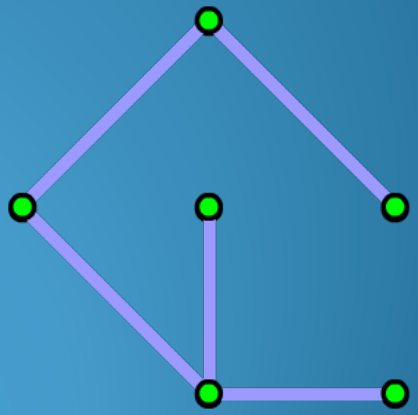
G2



G3



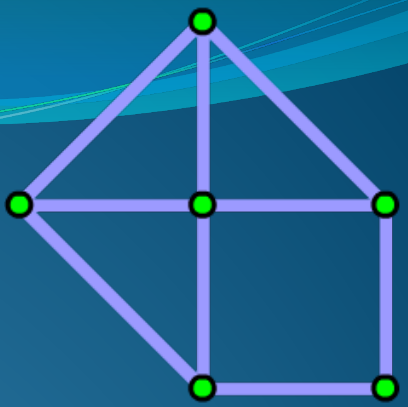
G4

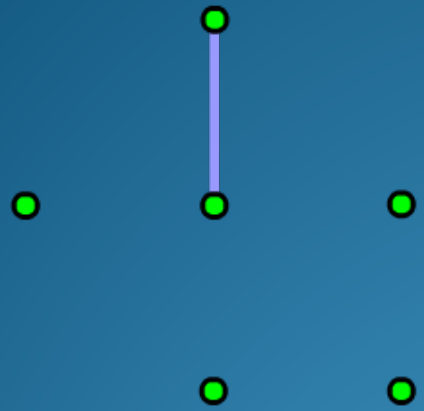


G5 = TG

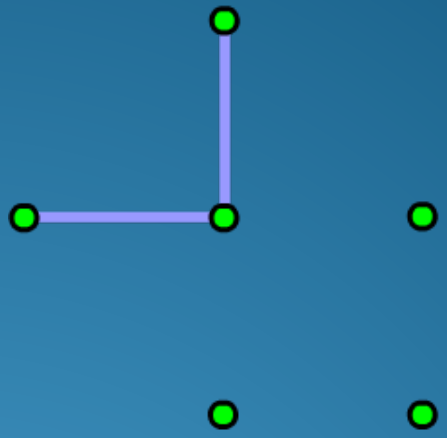
（b）避环法

用避环法构造（a）的生成树过程如下：



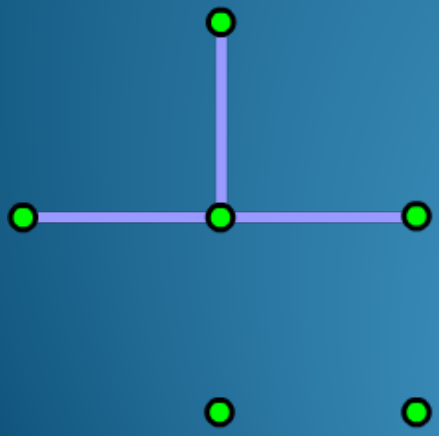


$G=G_1$

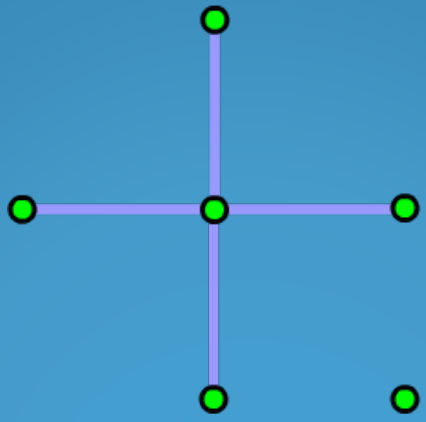


G_2

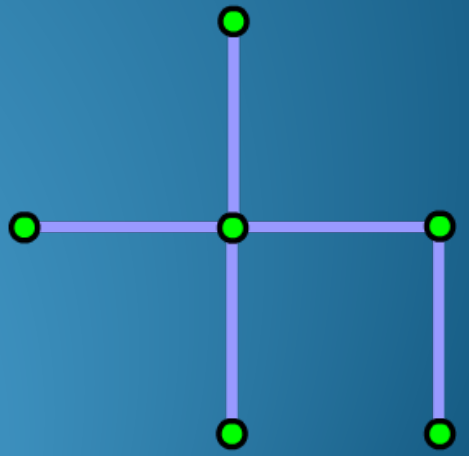
(a)



G_3



G_4

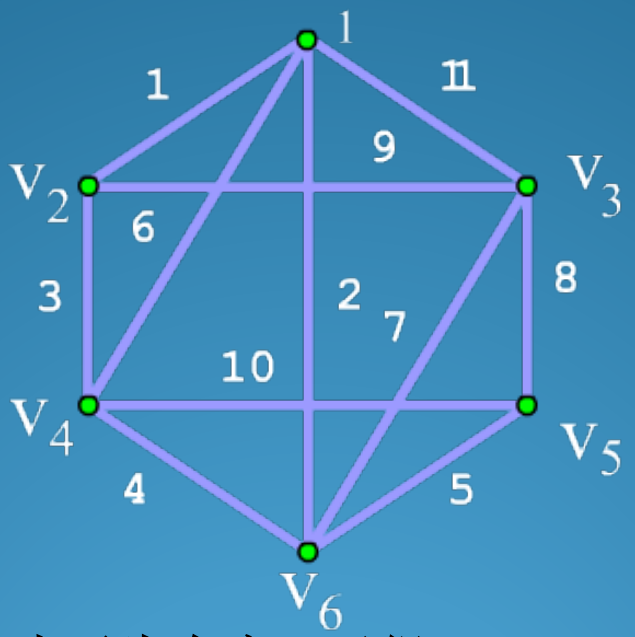


$G_1=T_G$

最小生成树

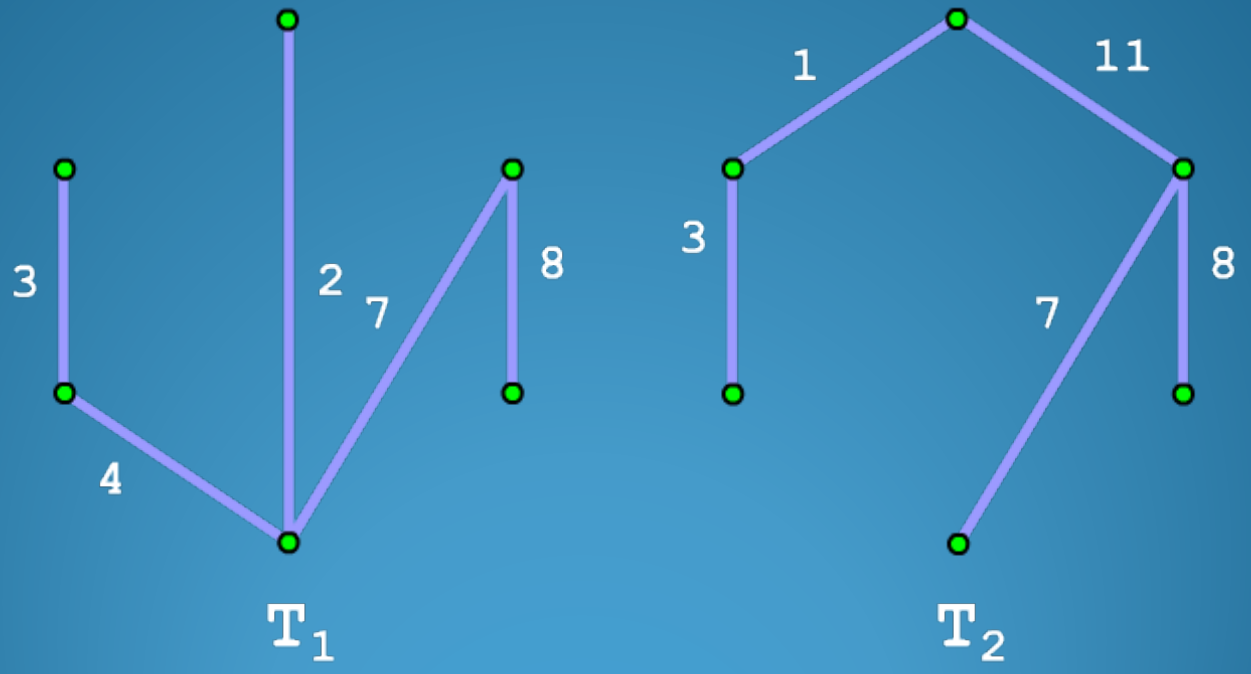
每条边都以实数赋权的图称为带权图。

下图是一连通带权图。



当G是一带权图时，常将其表示为有序三元组G=（V，E，f），这里f是一由边集E到实数集R的函数 f：E→R.

定义 设G=（V，E，f）是一连通带权图，T是G的一棵生成树，T的边集用E（T）表示，T的各边权值之和W（T）=$\sum_{e\in E(T)} f(e)$ 称为T的权。G的所有生成树中权最小的生成树称为G的最小生成树。

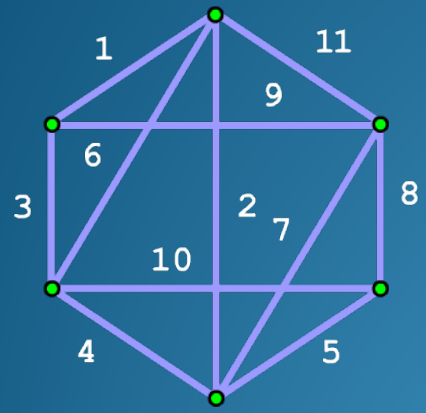


T_1 T_2

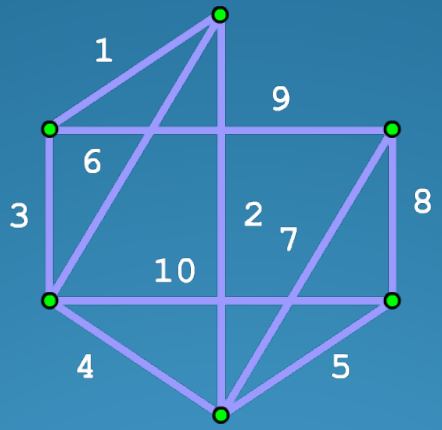
它们的权W（T_1）=24，W（T_2）=30。

构造连通带权图G=（V，E，f）的最小生成树的方法

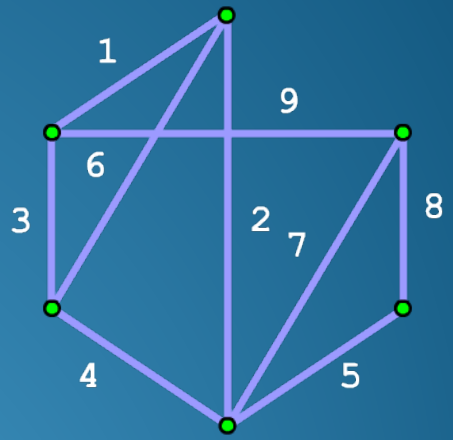
（1） 破环法



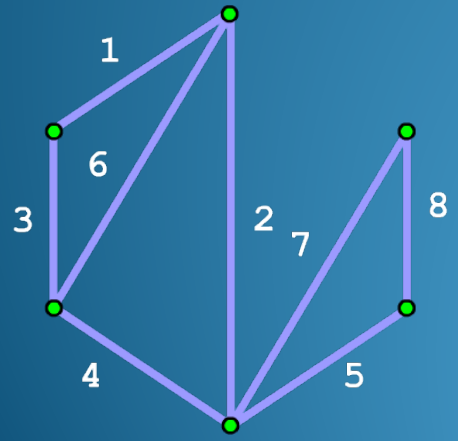
$G=G_1$



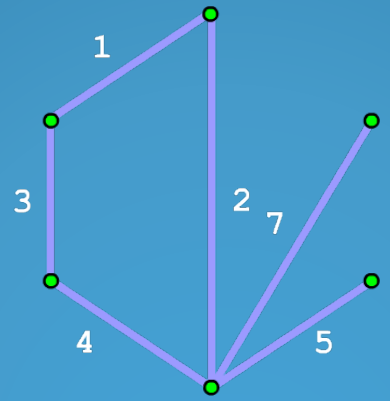
G_2



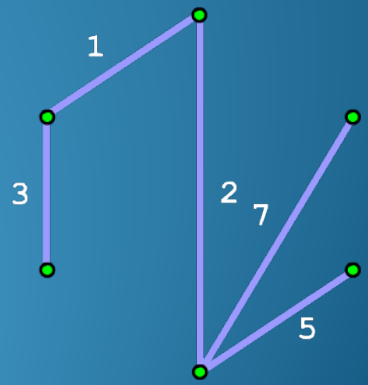
G_3



G_4

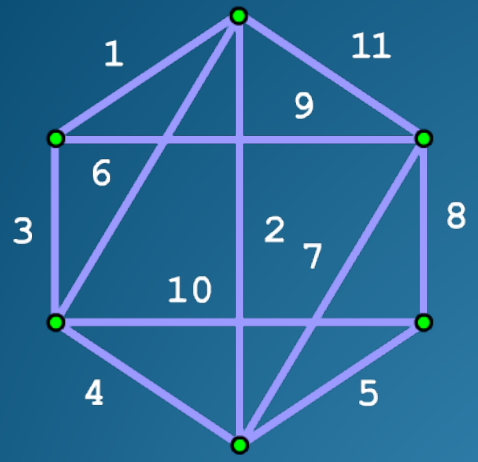


G_5

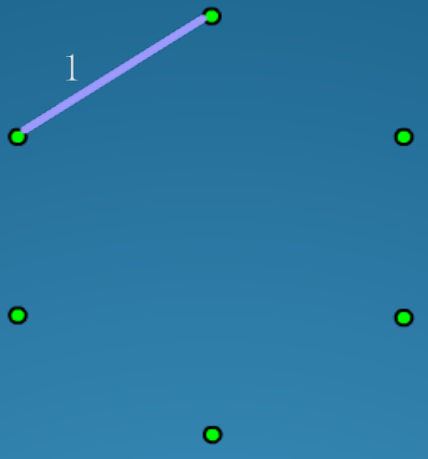


$W(T_0)=18$ $G_6=T_0$

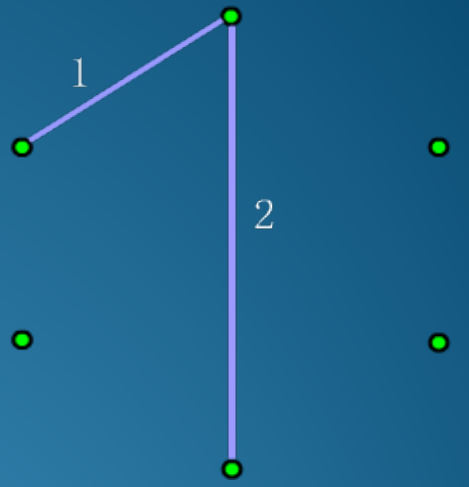
（2） 避环法



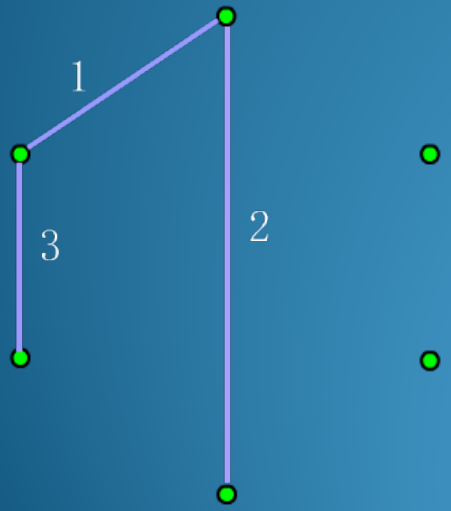
$G=G_1$



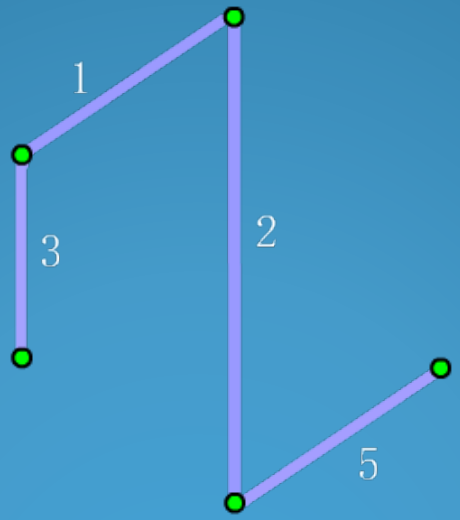
G_1



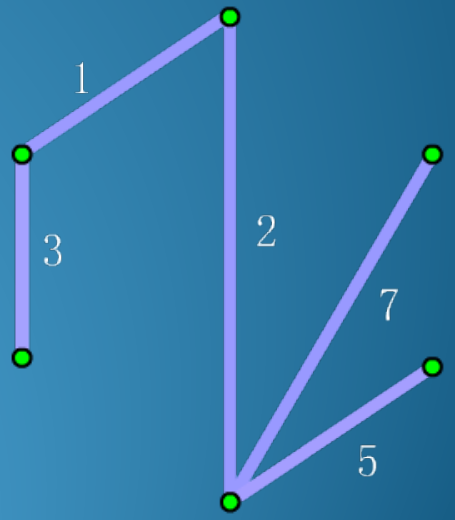
G_2



G_3



G_4



$G_5 = T_0$

深度优先搜索

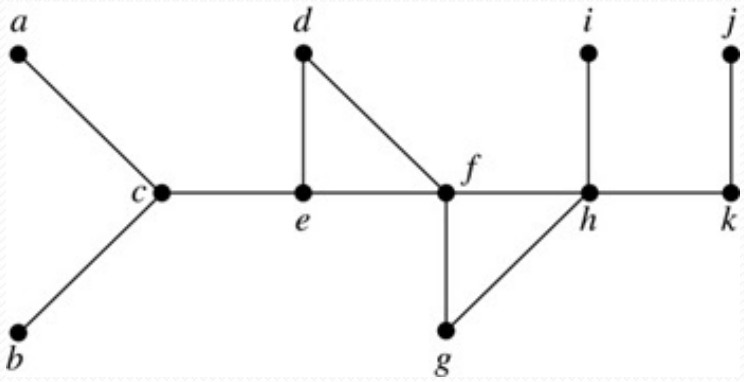
- 任意选择图中一个顶点作为根。通过相继地添加边来形成从这个顶点开始的通路，其中每条新边都与通路上的最后一个顶点以及还不在通路上的一个顶点相关联。继续尽可能地添加边到这条通路。若这条通路经过图的所有顶点，则由这条通路组成的树就是生成树。不过，若这条通路没有经过图的所有顶点，则必须添加其他的边。退到通路里的倒数第二个顶点，若有可能，则形成从这个顶点上开始的经过还没有访问过的顶点的通路。若不能这样做，则后退到通路里的另外一个顶点，即在通路里后退两个顶点，然后再试。重复这个过程，从所访问过的最后一个顶点开始，在通路上一次后退一个顶点，只要有可能就形成新的通路，直到不能添加更多的边为止。

Depth-First Search深度优先搜索

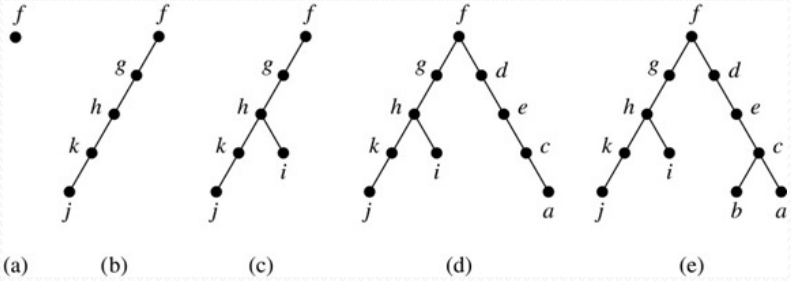
- To use *depth-first search* to build a spanning tree for a connected simple graph first arbitrarily choose a vertex of the graph as the root.
  - Form a path starting at this vertex by successively adding vertices and edges, where each new edge is incident with the last vertex in the path and a vertex not already in the path. Continue adding vertices and edges to this path as long as possible.
  - If the path goes through all vertices of the graph, the tree consisting of this path is a spanning tree.
  - Otherwise, move back to the next to the last vertex in the path, and if possible, form a new path starting at this vertex and passing through vertices not already visited. If this cannot be done, move back another vertex in the path.
  - Repeat this procedure until all vertices are included in the spanning tree.

Depth-First Search

Example: Use depth-first search to find a spanning tree of this graph.

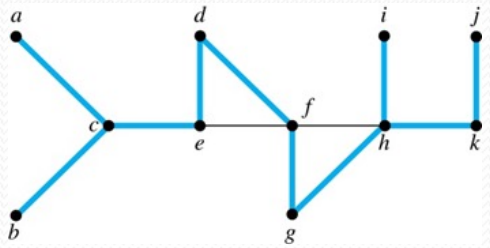


Solution: We start arbitrarily with vertex *f*. We build a path by successively adding an edge that connects the last vertex added to the path and a vertex not already in the path, as long as this is possible. The result is a path that connects *f*, *g*, *h*, *k*, and *j*. Next, we return to *k*, but find no new vertices to add. So, we return to *h* and add the path with one edge that connects *h* and *i*. We next return to *f*, and add the path connecting *f*, *d*, *e*, *c*, and *a*. Finally, we return to *c* and add the path connecting *c* and *b*. We now stop because all vertices have been added.



Depth-First Search (*continued*)

- The edges selected by depth-first search of a graph are called *tree edges*. All other edges of the graph must connect a vertex to an ancestor or descendant of the vertex in the graph. These are called *back edges*.
- In this figure, the tree edges are shown with heavy blue lines. The two thin black edges are back edges.



Depth-First Search Algorithm

- We now use pseudocode to specify depth-first search. In this recursive algorithm, after adding an edge connecting a vertex v to the vertex w, we finish exploring w before we return to v to continue exploring from v.

procedure *DFS*(G: connected graph with vertices $v_1, v_2, \ldots, v_n$)
T := tree consisting only of the vertex v_1
visit(v_1)

procedure *visit*(v: vertex of G)
for each vertex w adjacent to v and not yet in T
  add vertex w and edge $\{v, w\}$ to T
  visit(w)

Breadth-First Search

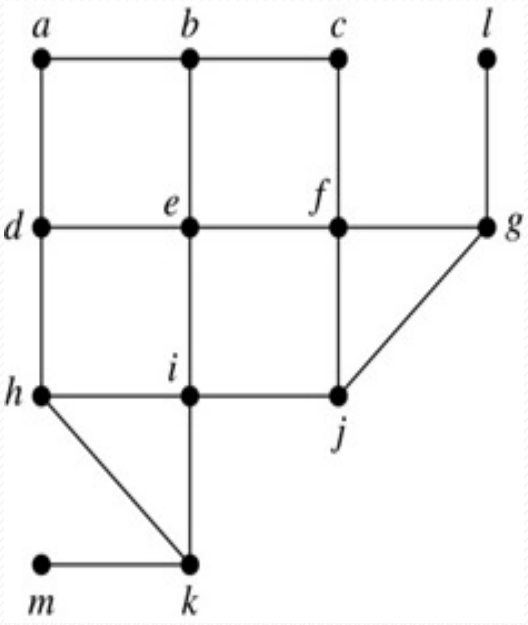
- We can construct a spanning tree using *breadth-first search*. We first arbitrarily choose a root from the vertices of the graph.
  - Then we add all of the edges incident to this vertex and the other endpoint of each of these edges. We say that these are the vertices at level 1.
  - For each vertex added at the previous level, we add each edge incident to this vertex, as long as it does not produce a simple circuit. The new vertices we find are the vertices at the next level.
  - We continue in this manner until all the vertices have been added and we have a spanning tree.

宽度优先搜索

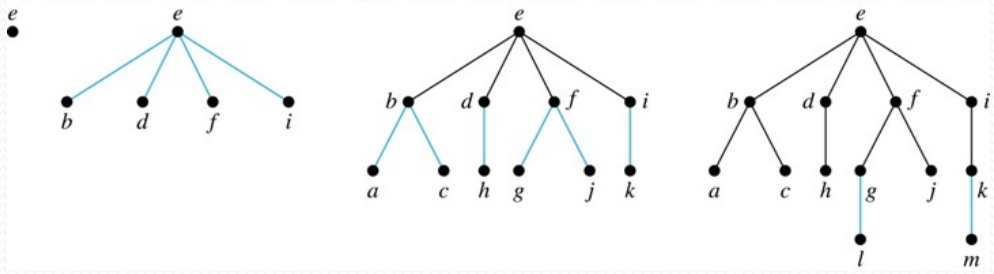
- 从图的顶点中任意地选择一个根。然后添加与这个顶点相关联的所有边。在这个阶段所添加的新顶点称为生成树在1层上的顶点。将新顶点任意排序。下一步，按顺序访问1层上的每个顶点，只要不产生简单回路，就将与这个顶点相关联的每条边添加到树里。这样就产生了树在2层上的顶点。遵循相同的过程，直到已经添加了树里的所有顶点。因为在图中的边数是有限的，所以这个过程会终止。在产生了包含图中每一个顶点的树之后，生成树也就产生了。

Breadth-First Search

Example: Use breadth-first search to find a spanning tree for this graph.



Solution: We arbitrarily choose vertex *e* as the root. We then add the edges from *e* to *b*, *d*, *f*, and *i*. These four vertices make up level 1 in the tree. Next, we add the edges from *b* to *a* and *c*, the edges from *d* to *h*, the edges from *f* to *j* and *g*, and the edge from *i* to *k*. The endpoints of these edges not at level 1 are at level 2. Next, add edges from these vertices to adjacent vertices not already in the graph. So, we add edges from *g* to *l* and from *k* to *m*. We see that level 3 is made up of the vertices *l* and *m*. This is the last level because there are no new vertices to find.



Breadth-First Search Algorithm

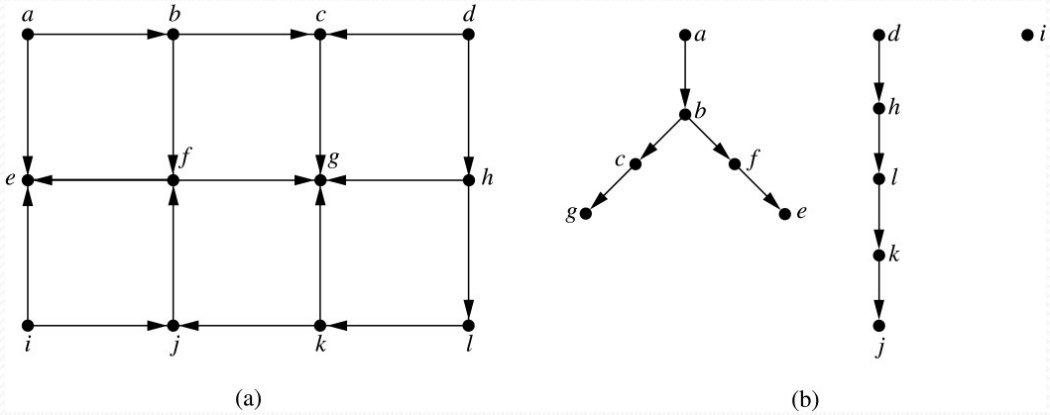
- We now use pseudocode to describe breadth-first search.

procedure *BFS*(*G*: connected graph with vertices $v_1, v_2, \ldots, v_n$)
T := tree consisting only of the vertex v_1
L := empty list *visit*(v_1)
put v_1 in the list L of unprocessed vertices
while L is not empty
    remove the first vertex, v, from L
    for each neighbor w of v
        if w is not in L and not in T **then**
            add w to the end of the list L
            add w and edge $\{v,w\}$ to T

Depth-First Search in Directed Graphs 有向图中的深度优先搜索

- Both depth-first search and breadth-first search can be easily modified to run on a directed graph. But the result is not necessarily a spanning tree, but rather a spanning forest.

Example: For the graph in (a), if we begin at vertex *a*, depth-first search adds the path connecting *a*, *b*, *c*, and *g*. At *g*, we are blocked, so we return to *c*. Next, we add the path connecting *f* to *e*. Next, we return to *a* and find that we cannot add a new path. So, we begin another tree with *d* as its root. We find that this new tree consists of the path connecting the vertices *d*, *h*, *l*, *k*, and *j*. Finally, we add a new tree, which only contains *i*, its root.



- To index websites, search engines such as Google systematically explore the web starting at known sites. The programs that do this exploration are known as *Web spiders*. They may use both breath-first search or depth-first search to explore the Web graph.

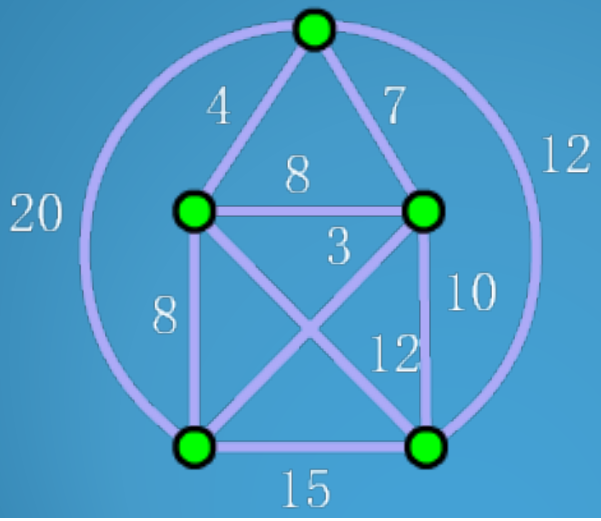
1．设树T有7条边，问T有多少个结点？（ 8 ）

2．设G是一个树林，由3个分图组成，若G有15个结点，问G有多少条边？（ 12 ）

3．互不同构的2结点树有（ 1 ）棵？

互不同构的4结点树有（ 2 ）棵？

4．求下图给出的带权图的最小生成树的权（24 ）



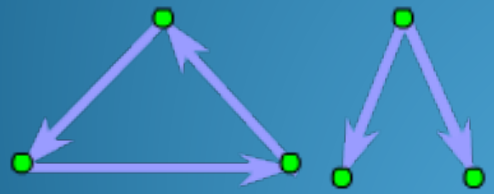
有向树

一、有向树的定义

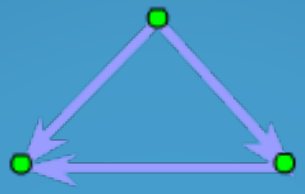
定义 一个不包含环的有向图G，若它只有一个结点v_0的入度为0，而其它所有结点的入度都等于1，则称G是有向树，v_0称为树的根。

一个孤立的结点也是一棵有向树。

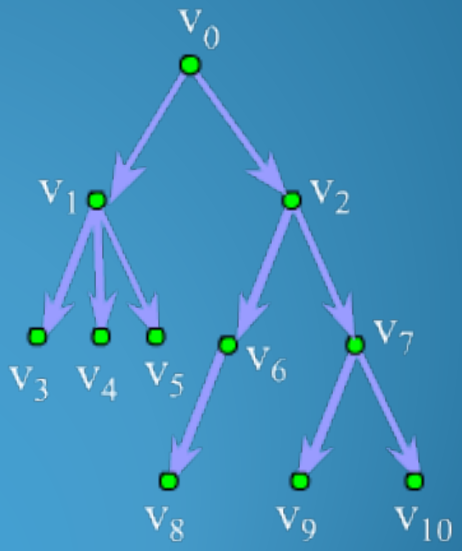
例 下 图给出的图是否有向树？



(a)



(b)



一层结点

二层结点

三层结点

二、有向树中的一些概念

（1）树叶

（2）内点

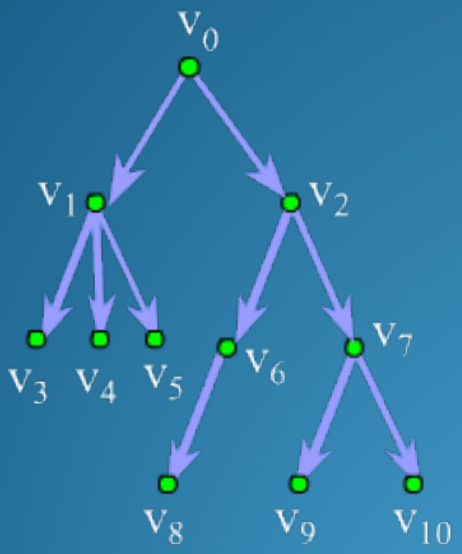
（3）儿子、父亲、兄弟、祖先和子孙（后代）

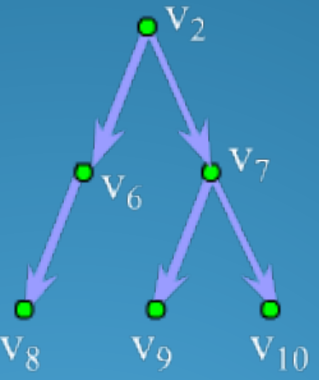
（4）以v为根的子树：设v是有向树T的分枝结点，由结点v和它的所有子孙构成的结点集V′，以及从v出发的所有有向路中的边构成的边集E′组成T的子图T′=（V′，E′）称为是以v为根的T的子树。

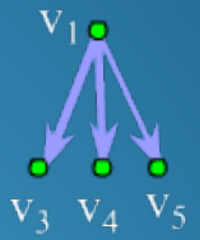
（5）v的子树：以v的儿子为根的子树。

子图$T_1=(V_1, E_1)$是以v_2为根的子树。其中

$V_1=\{v_2, v_6, v_7, v_8, v_9, v_{10}\}$，$E_1=\{(v_2, v_6), (v_2, v_7), (v_6, v_8), (v_7, v_9), (v_7, v_{10})\}$。







T_1又称为是v_0的子树。v_0的另一棵子树是以v_1为根的子树$T_2=(\{v_1, v_3, v_4, v_5\}, \{(v_1, v_3), (v_1, v_4), (v_1, v_5)\})$。

有向树中的一些数量关系

有向树和无向树一样，满足关系式 $m = n - 1$

定理　设T是一棵正则(完全)m元树，树叶结点数为n_0，内点数为t，则$(m-1)t=n_0-1$。

证明　由完全m元树的定义知，T的边数=mt，

结点数为 n_0+t，

于是　$mt=n_0+t-1$　　即　$(m-1)t=n_0-1$